

Advanced Deep Learning

Theory Lab 3

Christos Palantzidis

Task 3.1

Task 3.1.1: Explain the pros and cons of utilising Concatenation for combining embeddings

Concatenation is the most straightforward way to combine two embeddings: you just stick them together end-to-end. If your image vector has 2048 dimensions and your language vector has 512, the result is a single vector of 2560 dimensions. The next layer of the network then figures out how to make sense of the combined input on its own.

Pros

The biggest advantage is that nothing gets lost. Every value from both the image and the language side stays intact, and the network is free to learn which parts actually matter. You're not making any assumption upfront about how the two should interact — you just hand both to the next layer and let it sort things out through training.

Another practical benefit is that you don't have to worry about matching dimensions. With addition or multiplication, the two vectors must be the same size, so you usually end up adding a projection layer just to make them compatible. Concatenation skips that whole step. A 2048-dim ResNet feature and a 512-dim LSTM hidden state can be combined directly.

There's also flexibility in how the network treats each side. Because the weights connecting to the image dimensions are separate from those connecting to the language dimensions, the network can learn completely different transformations for each. With addition, both modalities go through the same weights, which is more restrictive.

On top of that, concatenation is dead simple to implement — one line in PyTorch (`torch.cat`) — and it tends to be a surprisingly strong baseline. On smaller datasets like Flickr8k, it often performs as well as or better than fancier approaches like attention, mainly because it doesn't introduce extra parameters that need lots of data to train properly.

Cons

The main downside is that the combined vector is bigger, which means the next layer needs more weights. If you're feeding the concatenated vector into a Dense layer with 1000 units, that's roughly 2.56 million parameters — almost double what addition would need. More parameters means more memory, slower training, and a higher risk of overfitting if your dataset isn't large.

Concatenation also doesn't give the network any built-in hint about how the two modalities should relate to each other. The model has to learn from scratch, purely from the data, that "dimension 47 of the image" corresponds in some way to "dimension 12 of the language vector." Other approaches bake in some kind of interaction by design — multiplication enforces a sort of agreement signal, attention enforces per-word focus on image regions — but concatenation just dumps everything together and hopes the next layer figures it out. With limited training data, that can be a problem.

Another issue is that there's no real interaction between the two vectors at the moment of combination. They sit side by side until the next linear layer mixes them. Compare that with multiplication, where the values from both sides immediately influence each other, or attention, where the language state actively selects which image regions matter.

Finally, when concatenation is used at the end of the pipeline (the so-called merge architecture), the language model never actually sees the image while it's encoding the partial caption. The image only enters at the very last step, right before the next word is predicted. Depending on the task, this can either help (cleaner separation between language modeling and visual grounding) or hurt (the image can't influence intermediate word choices).

When it makes sense

For a project like this one — Flickr8k, limited compute, a straightforward CNN-LSTM setup — concatenation is honestly a solid choice. It's reliable, easy to debug, and competitive with more complex methods at this scale. You'd start considering attention or other interaction-heavy approaches once you move to larger datasets like MS-COCO, where there's enough data to actually learn the extra parameters.

Task 3.1.2 - Explain the pros and cons of utilising Addition for combining embeddings

Addition is exactly what it sounds like: you take the image vector and the language vector and add them together element by element. If the image vector is $[2, 5, 1]$ and the language vector is $[4, 0, 3]$, the result is $[6, 5, 4]$. The combined vector stays the same size as each input — nothing grows. The catch is that for this to work at all, both vectors have to have the same number of dimensions, so you usually need a projection layer somewhere to make them match.

Pros

The most obvious win is efficiency. Unlike concatenation, addition keeps the vector size the same as the inputs, which means the next layer in the network has way fewer parameters to deal with. If you're working with 512-dim vectors, the next Dense layer sees 512 inputs instead of 1024, so memory usage and training time both go down. On smaller datasets where overfitting is a real concern, fewer parameters can actually translate to better generalization.

Addition also encourages the network to learn a shared semantic space for both modalities. To make the sum meaningful, the model has to push image features and language features into compatible regions of the vector space — basically forcing a kind of alignment between "what the image says" and "what the words say." This can lead to more transferable representations and is conceptually clean.

Another advantage is how nicely it plays with gradient-based training. Addition is the same basic operation that powers residual connections in ResNets, and we know those train really well. Gradients flow back equally to both inputs without any weird amplification or suppression. There's nothing exotic happening mathematically, so optimization tends to be stable.

It's also symmetric — neither modality gets architectural priority over the other. Both contribute equally to every dimension of the output, which feels right when you don't have a strong reason to favor one source over the other.

Cons

The biggest problem with addition is information cancellation. If the image vector contributes $+0.7$ to some dimension and the language vector contributes -0.7 , the sum is exactly zero. The next layer has no way of knowing whether that dimension was just uninformative to begin with, or whether two strong but opposing signals just cancelled each other out. That's a real loss of information that concatenation simply doesn't have, since concatenation keeps both values around for the network to inspect.

The dimension-matching requirement is another headache. You can't just add a 2048-dim ResNet feature to a 512-dim LSTM state directly — you have to project one of them first. That projection layer adds parameters of its own and, more importantly, acts as a bottleneck: you're compressing 2048 dimensions of image information down to 512 before the addition even happens. Some of the original detail from the image is gone before you've even started combining things.

Addition also makes a pretty strong assumption that doesn't always hold. It assumes the two modalities live in the same vector space and that adding them produces a meaningful blend. But pixel-derived features and word-derived features are fundamentally different kinds of representations, and forcing them to add coherently is a real constraint on what the model can learn. Concatenation makes no such assumption — it just hands both representations to the next layer and lets the network figure out the relationship.

There's also a loss of expressive power compared to concatenation. With addition, the downstream layer applies the *same* set of weights to a sum of contributions, so it can't treat image-derived information differently from language-derived information at that step. Concatenation lets the network learn separate weights for each side, which gives it more flexibility in how it processes each modality.

When it makes sense

Addition is a good fit when you genuinely want a shared embedding space — say, in retrieval tasks where you're matching images to captions, or in models where the symmetry between modalities

matters. It's also a reasonable choice when you're worried about parameter count or working with very small datasets. But for image captioning specifically, the cancellation issue and the dimension-matching constraint often make concatenation a safer bet, especially as a starting baseline. Addition tends to shine more in setups where the two vectors have already been carefully aligned through training (e.g., CLIP-style models), rather than as a quick fusion step in a CNN-LSTM pipeline.

Task 3.1.3 - Explain the pros and cons of utilising Multiplication for combining embeddings

Multiplication, in this context, almost always means the **element-wise product** (also called the Hadamard product). You take the image vector and the language vector and multiply them position by position. So if the image vector is $[2, 5, 1]$ and the language vector is $[4, 0, 3]$, the result is $[8, 0, 3]$. Like addition, the output is the same size as each input, and like addition, this only works when both vectors have the same number of dimensions, so a projection layer is usually needed to make them match.

Pros

The interesting thing about multiplication is that it captures *interaction* between the two modalities, not just superposition. With addition, both signals coexist in the output. With multiplication, a dimension only "lights up" when both the image and the language side have meaningful values at that position. This gives the network a kind of agreement signal — output values are large only when both modalities agree that something is important. That's a fundamentally different kind of combination than what addition or concatenation offer.

This same property makes multiplication act as a natural gating mechanism. The language vector can effectively select which dimensions of the image features matter at any given moment by zeroing out or amplifying specific positions. This is exactly the same operation that powers LSTM gates, GRU gates, and gated linear units, all of which are well-established and known to work. Conceptually, it fits image captioning quite well — you want the current word context to influence which image features are relevant for predicting the next word.

It's also parameter-free at the combination step itself. You don't add any new weights when multiplying the two vectors together (apart from whatever projection you needed to match dimensions in the first place). The output stays the same size as the inputs, so the downstream layers don't blow up in size the way they do with concatenation.

Cons

The biggest issue with multiplication is that zeros are absorbing. If either the image or the language vector has a near-zero value in some dimension, the product in that dimension is also near zero, and any signal the *other* modality wanted to contribute is just gone. This is a much harsher form of information loss than what addition does. With addition, two strong opposing values can cancel out,

but at least each side had a chance to contribute. With multiplication, one side can completely silence the other, no matter how strong the other side's signal was.

Training stability is another concern. Multiplicative interactions tend to make gradient magnitudes more variable than additive ones. During backpropagation, the gradient for each branch is scaled by the activation value of the *other* branch, which means gradients can get amplified or suppressed in ways that depend on the current state of the model. Without careful normalization (LayerNorm, BatchNorm, or similar), this can lead to instability, especially early in training when activations haven't settled.

Like addition, multiplication requires matched dimensions, so you're stuck adding a projection layer and accepting whatever bottleneck that creates. You also lose the ability to keep image and language information cleanly separated — once they're multiplied, you can't really tell which dimensions came from where, and the downstream layer has less flexibility to treat them differently.

There's also an empirical point worth mentioning. In direct comparisons on standard captioning benchmarks (like in the Tanti & Gatt paper), Hadamard multiplication tends to underperform concatenation as the top-level fusion step. It works much better when used *inside* an attention block — where multiplication appears naturally as part of computing attention scores — than as a standalone way to merge image and language vectors at the end of the pipeline.

When it makes sense

Multiplication is the right choice when you specifically want a gating or agreement signal between modalities — for example, in visual question answering, where you want the question to filter which parts of the image are relevant. It also shows up all over the place inside attention mechanisms, where it does its best work. As a top-level fusion operator for image captioning, though, it's usually not the strongest pick. Concatenation is more forgiving and addition is more stable, so multiplication tends to be more useful as a building block inside more complex architectures than as the main combination step on its own.

Task 3.1.4 - Explain the pros and cons of utilising Attention for combining embeddings

Attention works fundamentally differently from the other methods on this list. Instead of squashing the entire image into a single feature vector and then combining it with the language vector, attention keeps the image as a *grid* of features — for example, the $14 \times 14 \times 512$ output you get from a CNN before the final pooling step. That gives you a set of regional feature vectors, one per spatial location. Then, at each step of generating the caption, the model computes a weighted combination of these regional features, where the weights depend on the current state of the language model. Different words get different weightings, so the model can focus on different parts of the image as it generates the caption.

Concretely, when generating the word "frisbee," the model can put most of its weight on the region containing the frisbee. When generating "boy," it can shift its focus to the region containing the boy. The weights are computed by a small learned scoring function (Bahdanau-style additive attention, Luong-style multiplicative, or scaled dot-product as in transformers). This is the architecture behind the well-known "Show, Attend and Tell" paper by Xu et al., and it's the foundation of pretty much every modern captioning system.

Pros

The big advantage is dynamic, per-word visual grounding. With concatenation, addition, or multiplication, the entire image gets compressed into one vector, and that same vector is used for every single word in the caption. That's a real bottleneck — the word "the" doesn't need the same visual context as the word "frisbee," but a static fusion method can't tell the difference. Attention removes this bottleneck completely. Each word gets its own custom view of the image, computed on the fly based on what's already been generated.

Performance reflects this. Attention-based models consistently outperform static fusion methods on the standard benchmarks (MSCOCO, Flickr30k), often by a substantial margin. Pretty much every state-of-the-art captioning system in the last several years uses attention in some form, and the move toward transformer-based decoders has only made attention more central.

Attention also gives you something the other methods don't: interpretability. You can take the attention weights for each generated word and overlay them on the image to see exactly where the model "looked" while producing that word. This is genuinely useful for debugging — if your model says "dog" but the attention map shows it focused on the sky, something is wrong. For end users, especially in safety-critical applications, this kind of visual evidence builds trust in a way that black-box methods can't.

It also handles variable spatial resolution naturally. You're not forced to commit to a single global pooling of the image; the model just operates over whatever grid the CNN produces, and the spatial structure is preserved through the attention step. And the same machinery scales straight to transformers, which is where modern captioning has converged. So building on attention isn't just a current win — it's also the right foundation for moving forward.

Cons

The main cost is, well, cost. Attention is computationally heavier than static fusion. For each word you generate, you have to compute attention scores against every spatial location in the image, which adds up. Memory and FLOPs both grow with the image's spatial resolution, and longer captions multiply the work. On smaller setups or when training time matters, this overhead is real.

There are also more parameters. The scoring function brings its own learnable weights on top of the encoder and decoder, and depending on the variant (especially multi-head attention), the parameter count can grow significantly. More parameters mean you need more training data to learn meaningful weights without overfitting. On a small dataset like Flickr8k, a poorly tuned attention model can actually underperform a well-tuned concatenation baseline, which is a counterintuitive result but shows up in practice.

Implementation complexity is another factor. Attention has more moving parts than `torch.cat` — you need to handle masking, normalization, broadcasting, and the right reshaping of tensors. There are more places for subtle bugs to hide, and debugging is harder because the failure modes are less obvious. If your loss isn't decreasing, is it your attention layer? Your normalization? Your masking? It takes more effort to track down.

Early training can also be slow. Attention weights start out essentially random, so for the first few epochs the model is "looking" at the wrong parts of the image, which slows convergence. Some papers report that warmup tricks or curriculum learning help, but it's an extra layer of effort that you don't need with simpler fusion methods.

One more thing worth being honest about: attention maps are not always as clean as they look in published papers. The figures you see in papers tend to be cherry-picked, and in practice attention can be diffuse, focused on weird background regions, or simply not align well with what humans would call relevant. So while attention is interpretable in principle, the actual maps you get for any given example might not tell a clean story.

When it makes sense

Attention is the right choice when you have enough data, enough compute, and care about top-line performance. For MSCOCO-scale datasets and modern captioning systems, it's basically the default — there's no good reason not to use it. For something like Flickr8k with a CNN-LSTM backbone, the calculus is less obvious. Attention can still help, but a well-tuned concatenation baseline might come surprisingly close while being much simpler to implement and debug. If you're going for the strongest possible result and you have the budget for it, attention is the way. If you're prioritizing simplicity, reliability, and quick iteration, static fusion is still very competitive at smaller scales.

Task 3.1.5 - Explain the pros and cons of utilising Difference for combining embeddings

Difference combines two embeddings by subtracting one from the other, element by element. So if the image vector is $[2, 5, 1]$ and the language vector is $[4, 0, 3]$, the result is $[-2, 5, -2]$. Sometimes the absolute value is taken instead, giving $[2, 5, 2]$, which makes the operation symmetric (the order of subtraction stops mattering). Like addition and multiplication, this requires both vectors to have the same dimensionality, so a projection layer is usually needed first. The output stays the same size as the inputs, which keeps things parameter-efficient at the combination step itself.

Pros

The main thing difference is good at is highlighting *discrepancy* between two inputs. Where addition produces a blend and multiplication produces an agreement signal, subtraction produces a

contrast signal. Each output dimension tells you not how much both modalities agree, but how much they disagree. That can be exactly what you want in tasks where the relevant information is the gap between two sources — visual entailment (does this caption contradict this image?), change captioning (what's different between these two images?), contrastive learning, or anomaly detection. In those settings, difference is a clean and direct way to extract the contrast.

It's also parameter-free at the combination step, just like addition and multiplication. No new weights are introduced when you subtract two vectors, so it's cheap.

Another point in its favor is that it pairs well with addition. If you compute both $v_{\text{img}} + v_{\text{lang}}$ and $v_{\text{img}} - v_{\text{lang}}$ and concatenate them, you get a representation that captures both shared and contrastive information at the same time. This kind of (sum, difference) decomposition is used in some siamese architectures and in certain matching networks, and it's a reasonable way to get more out of the two vectors than either operation alone provides.

The absolute-value version ($|v_{\text{img}} - v_{\text{lang}}|$) has the additional benefit of being symmetric — neither modality is privileged over the other. This matches the inductive bias you'd want in cases where the relationship between the two sources is genuinely undirected.

Cons

The fundamental problem is that difference is the wrong inductive bias for image captioning. In captioning, the image and the caption aren't supposed to disagree — the caption is meant to *describe* the image. The shared content between the two modalities is exactly the signal you want to keep, and difference deletes it. Subtracting a language vector that correctly represents the image from the image vector itself would, in the limit of perfect grounding, give you a vector close to zero. The better your model gets at aligning the two modalities, the less signal the difference operation produces. That's the opposite of what you want.

There's also a vanishing-signal problem connected to this. As training progresses and the model learns to map matching image-caption pairs into similar regions of the vector space (which is the goal of grounded language learning), the difference between them shrinks. So just when the model is starting to work, the combined representation gets weaker. This is a strange dynamic to deal with during training and tends to make difference unreliable as a standalone fusion step.

The non-absolute version is asymmetric in an arbitrary way. Why is it $v_{\text{img}} - v_{\text{lang}}$ and not $v_{\text{lang}} - v_{\text{img}}$? There's usually no principled reason to pick one over the other, and the choice silently introduces an architectural bias that doesn't reflect anything meaningful about the task. The absolute-value version fixes this but introduces its own issue — taking the absolute value is non-differentiable at zero and can cause optimization quirks if not handled carefully.

It also shares the dimension-matching constraint with addition and multiplication. You need a projection layer to bring both vectors to the same size, with all the bottleneck issues that come with that.

Finally, there's the empirical reality: difference is essentially never used as the *sole* combination operator in published image captioning systems. When it appears at all, it's as an auxiliary feature

concatenated alongside other operations — for example, in some VQA architectures you might see `[v_img; v_lang; v_img * v_lang; |v_img - v_lang|]` all concatenated together as a rich joint representation. But on its own, as the way to merge image and text in a captioning pipeline, it's not a serious candidate.

When it makes sense

Difference makes sense for tasks where contrast *is* the signal — comparing two images, detecting contradictions between text and image, or building contrastive representations. For straight image captioning, it's the weakest of the five options on the list, because the operation actively discards the part of the signal you most want to keep. The most defensible use of difference in a captioning context is as one component of a richer joint representation alongside concatenation or other operations, not as a replacement for them.

References

- [1] Tanti, M., Gatt, A., & Camilleri, K. P. (2018). Where to put the Image in an Image Caption Generator. *Natural Language Engineering*, 24(3), 467–489. <https://doi.org/10.1017/S1351324918000098>
- [2] Tanti, M., Gatt, A., & Camilleri, K. P. (2017). What is the Role of Recurrent Neural Networks (RNNs) in an Image Caption Generator? In *Proceedings of the 10th International Conference on Natural Language Generation (INLG)*, 51–60.
- [3] Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhutdinov, R., Zemel, R., & Bengio, Y. (2015). Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2048–2057.

Github link : <https://github.com/chris1982-arch/labs-theory>